

Docker

Major Scenario

Suppose I'm building an **e-commerce application** using React.js for the frontend and Node.js 18 for the backend. Alongside I'm using redis for caching and postgres for database etc.

Now after a few months of development, I want to hire a new developer to join the team. So at that point of time the other developer might not have the same environment as mine. Maybe Node.js 18 is not installed on their machine, or maybe they are using a different version of Node.js. This inconsistency can lead to bugs and issues that are hard to debug.

Later if I add another tool to add another feature to my application, I need to ensure that the new tool is compatible with the existing tools and that the application still runs smoothly in the other developer's machine, and also in the production environment.

- I want to ensure that the application runs consistently across different environments, such as development, testing, and production. How can Docker help me achieve this?

What problem is Docker trying to solve ?

- Docker is trying to solve the problem of "it works on my machine" by providing a consistent environment for development and deployment.
- It allows developers to package an application with all of its dependencies into a standardized unit for software development.
- Docker containers wrap a piece of software in a complete filesystem that contains everything needed to run: code, runtime, system tools, system libraries – anything that can be installed on a server.
- This guarantees that the application will always run the same, regardless of where it is deployed.

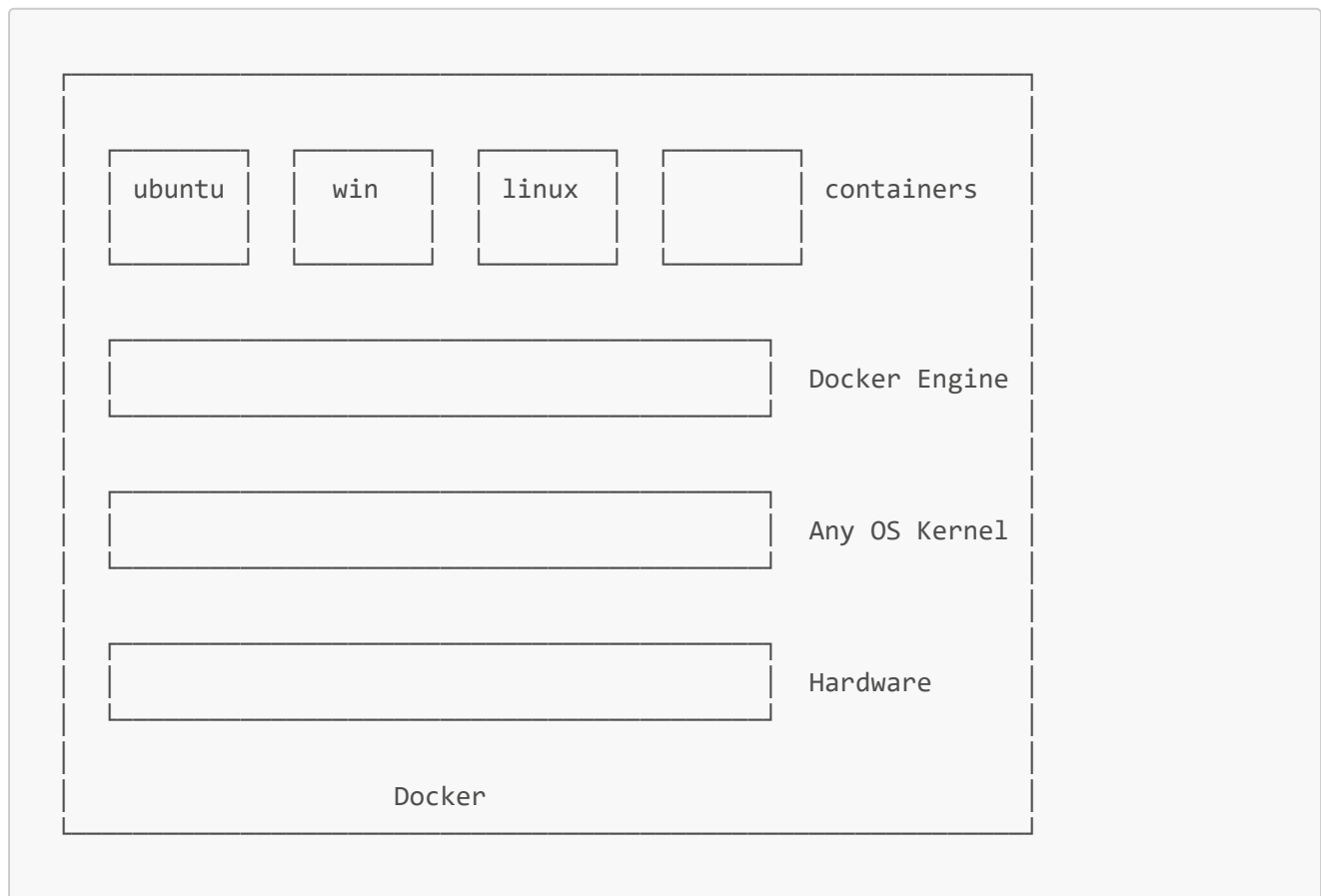
Alternate Solutions to Docker

- **Virtual Machines (VMs):**

I've to install some kind of hypervisor like VMware, VirtualBox, or Hyper-V to run VMs. Install ubuntu or any other OS on the VM and then install the required software on the VM. This is a heavy solution and takes a lot of time to set up. I've to make sure that my other developer has the same VM setup as mine & in the production I've to make sure that the VM is running on the same OS as mine.

✳️ Cons:

- * Heavyweight
- * Slow to start
- * Resource intensive
- * Not portable



Docker runs on top of the OS Kernel and uses the OS kernel to run the containers. So docker is not a virtual machine. It is a containerization platform. Whatever OS you are using, docker will run on top of it.

- Virtual Machine you have to install the OS and then install the software on the OS but in Docker you don't have to install the OS. You just have to install the docker and then you can run the containers on top of the OS kernel.
- In VMs you're using full OS but in Docker you're using only the OS kernel. Hence Docker is lightweight and fast. In docker's word we call each of slice of these OS environment as a image.

Install Docker

- Docker Desktop for Windows and Mac
- Docker Engine for Linux
- Docker Toolbox for older Mac and Windows systems

Check docker version: `docker version`

Docker Ubuntu Install

```
docker run -it ubuntu
```

- `docker run` - Run a command in a new container
- `-it` - Interactive terminal

- `ubuntu` - Image name
- `bash` - Command to run in the container

This will download the ubuntu image from the docker hub and run it in a container. You can run any command in the container. For example, you can run `apt-get update` to update the package list.

Dockerization

Dockerization is the process of packaging an application and its dependencies into a Docker container.